

AUDIT TECHNIQUE CONFIDENTIEL

Rapport d'audit du projet Infinity WAB

Analyse de structure, performance, sécurité et vulnérabilités

Projet : infinitywab_web — Application Laravel 11 / Vue 3 / Vite

Répertoire : d:\NAZAIRE\ENTP\SOFT_DEV\DEV\infinitywab_web

Date du rapport : 16 juillet 2026

Méthode : revue statique du code source (analyse hors exécution, lecture seule)

Table des matières

1. Synthèse exécutive
2. Structure du projet et stack technique
3. Dépendances et versions
4. Sécurité et vulnérabilités
 - 4.1 CSRF désactivé sur l'API d'administration
 - 4.2 APP_KEY vide en environnement réel
 - 4.3 Fallback d'admin par email en dur
 - 4.4 XSS stocké potentiel (contenu non sanitisé)
 - 4.5 innerHTML non échappé côté admin
 - 4.6 Absence de headers de sécurité HTTP
 - 4.7 Confiance globale envers les proxys
 - 4.8 Route de test morte
 - 4.9 Points positifs constatés
5. Performance
6. Qualité, tests et manquements (CI/CD, lint, doc)
7. Tableau de synthèse des priorités
8. Plan de remédiation recommandé

1. Synthèse exécutive

Infinity WAB est un site vitrine développé en **Laravel 11 (PHP 8.4)** avec un rendu Blade côté serveur, un espace d'administration protégé par session, et une base d'outillage Vue 3 / Vite / TypeScript installée mais très peu exploitée dans le code actuel. L'architecture est globalement saine (usage systématique d'Eloquent, pas d'injection SQL brute détectée, validation d'entrée présente sur les contrôleurs admin), mais l'audit révèle **deux problèmes de sécurité de sévérité haute** qui doivent être corrigés avant toute mise en production : la désactivation de la protection CSRF sur l'ensemble de l'API d'administration, et une clé d'application (APP_KEY) vide qui empêche le chiffrement correct des sessions et cookies.

S'y ajoutent plusieurs manquements de sévérité moyenne (fallback d'administrateur codé en dur, absence de sanitisation HTML sur du contenu affiché publiquement, absence de headers de sécurité HTTP, serveur de développement utilisé comme processus de production) et des lacunes de qualité logicielle (suite de tests non fonctionnelle, absence de CI/CD, configuration de lint manquante). Aucune de ces découvertes ne relève d'une compromission active constatée : il s'agit d'un audit de code statique destiné à durcir le projet avant/pendant son exploitation.

2. Structure du projet et stack technique

Backend — PHP 8.4, Laravel 11, ORM Eloquent

Frontend rendu — Blade server-side (vues .blade.php)

Build — Vite 7 + laravel-vite-plugin, Tailwind CSS 3.4, Sass

Déploiement — Nixpacks / Railpack (type Railway/Dokploy), pas de Docker

Auth — laravel/ui 4.6 (scaffolding Bootstrap/Blade)

Frontend outillage — Vue 3.5 + Vue Router 4 installés, quasi inutilisés

Base de données — MySQL

Gestion de versions — pas de dépôt git initialisé actuellement

Arborescence principale observée :

```
app/Http/Controllers/{Admin/**, Auth/**, *.php}    contrôleurs publics + admin (Web + Admin/Api
JSON)
app/Http/Middleware/AdminMiddleware.php
app/Models/                                         Company, Content, Message, Partner, Product, Project, Service, User
+ 3 fichiers vides : ServiceResource.php, StoreServiceRequest.php,
UpdateServiceRequest.php
app/Notifications/VerifyEmailAntiSpam.php
bootstrap/app.php                                configuration middleware/routing (style Laravel 11)
config/                                             app, auth, cache, database, filesystems, logging, mail, queue, session
(pas de config/cors.php ni config/sanctum.php)
database/migrations, seeders, factories
resources/views/                                   home, services, projects, products, company, contact, about, auth/*,
admin/*
resources/js/, resources/css/, resources/sass/
routes/web.php, routes/console.php (pas de routes/api.php)
tests/Feature, tests/Unit 2 fichiers de test génériques, non représentatifs
scripts/deploy-sans-docker.sh
public/index.php, .htaccess, favicon.ico, robots.txt
```

Fichiers de configuration clés : `.env` / `.env.example` , `package.json` , `composer.json` / `composer.lock` , `vite.config.js` , `tailwind.config.js` , `phpunit.xml` , `nixpacks.toml` , `railpack-plan.json` . Aucun `Dockerfile` , aucun dossier `.github/workflows` : **pas de pipeline CI/CD**.

3. Dépendances et versions

Écosystème	Dépendance	Version	Constat
PHP	laravel/framework	^11.0	Récente, pas de CVE flagrante identifiée
PHP	laravel/ui	^4.6	OK
PHP (dev)	phpunit/phpunit, laravel/pint	^11.5 / ^1.24	Installés mais peu exploités (voir §6)
JS	vue, vue-router	^3.5.29 / ^4.6.4	Quasi inutilisés dans le code réel — dette technique

JS (dev)	eslint	^8.45.0	Fin de vie (EOL), non maintenu depuis fin 2024
JS (dev)	vite	^7.0.7	OK
JS (dev)	typescript, vue-tsc, vitest	diverses	Installés, aucun fichier .ts/.vue/.vitest trouvé — poids mort

`composer.lock` et `package-lock.json` sont présents (bon point pour la reproductibilité des builds). Un `composer audit` et un `npm audit` réels n'ont pas été exécutés dans le cadre de cette revue statique — ils sont recommandés en complément (voir §8).

4. Sécurité et vulnérabilités

SÉVÉRITÉ HAUTE 4.1 — CSRF désactivé sur toute l'API d'administration

`bootstrap/app.php`, ligne ~18

```
$middleware->validateCsrfTokens(except: [  
    'api/admin/*',  
]);
```

Toutes les routes `POST/PUT/DELETE` de `api/admin/*` (création/suppression de services, produits, projets, utilisateurs admin, messages, partenaires, mise à jour de la société) sont exemptées de vérification CSRF alors qu'elles reposent sur une authentification par **cookie de session** (middleware `admin + verified`), et non sur un token API dédié. C'est une désactivation globale de la protection CSRF sur la surface d'attaque la plus sensible de l'application — y compris la gestion du drapeau `is_admin`. `SESSION_SAME_SITE=lax` atténue partiellement le risque mais ne le neutralise pas.

Recommandation : retirer `api/admin/*` de l'exception CSRF, ou migrer vers Laravel Sanctum (mode SPA) avec vérification CSRF standard sur les routes d'administration.

SÉVÉRITÉ HAUTE 4.2 — APP_KEY vide dans l'environnement réel

`.env`, ligne ~10

Le fichier `.env` réel présent sur le projet a un `APP_KEY` vide (jamais généré via `php artisan key:generate`), ainsi que `DB_PASSWORD` et les identifiants mail. Sans `APP_KEY`, le chiffrement des sessions et des cookies signés est cassé, ce qui provoquera des erreurs au runtime (ou un chiffrement absent selon la configuration) — inacceptable en production.

Recommandation : générer une clé d'application dédiée par environnement, ne jamais versionner `.env` réel, valider systématiquement au démarrage que les secrets requis sont renseignés.

SÉVÉRITÉ MOYENNE**4.3 — Fallback d'administrateur basé sur un email en dur**

app/Http/Middleware/AdminMiddleware.php, ligne 27 ·
app/Http/Controllers/Auth/LoginController.php, ligne 17

```
$isAdmin = (bool) ($user->is_admin ?? false) || ($user->email === 'admin@infinity-wab.bf');
```

Un compte portant cette adresse email précise obtient automatiquement les droits admin, indépendamment du champ `is_admin` en base. Si ce compte est recréé, repris, ou si la vérification d'email est contournée, cela constitue une porte dérobée implicite dupliquée à deux endroits du code.

Recommandation : supprimer ce fallback et se reposer exclusivement sur le flag `is_admin`.

SÉVÉRITÉ MOYENNE**4.4 — XSS stocké potentiel via contenu non sanitisé**

resources/views/service-detail.blade.php:101 · project-detail.blade.php:81 · product-detail.blade.php:100

```
{!! $service->content !!}
```

Le champ `content` (texte libre, validé uniquement en `nullable|string` côté contrôleurs Admin/Api, sans sanitisation HTML) est affiché sans échappement sur des pages publiques. Aucune bibliothèque de sanitisation (type HTMLPurifier) n'est utilisée dans le projet. Le risque direct concerne un compte admin compromis ou malveillant qui pourrait injecter du HTML/JS exécuté côté visiteurs, voire pour élever ses privilèges vers d'autres comptes admin via vol de session.

Recommandation : sanitiser le HTML riche à l'enregistrement (ex. HTMLPurifier) ou restreindre ce champ à du Markdown converti côté serveur avec une liste blanche de balises.

SÉVÉRITÉ BASSE/MOYENNE**4.5 — innerHTML non échappé dans les vues d'administration**

resources/views/admin/products.blade.php:264-291 · admin/projects.blade.php:~228-288 ·
admin/services.blade.php:~197-253

Une fonction `escapeHtml()` existe et est correctement utilisée pour les messages de contact (saisis par des visiteurs anonymes, source non fiable) dans `admin/dashboard.blade.php:141`, mais elle n'est pas réutilisée dans les listings admin de produits/projets/services, où les données sont injectées brutes via `innerHTML`. Risque plus faible (données censées être saisies par des admins) mais incohérent avec la bonne pratique déjà en place ailleurs, et aggrave l'impact du point 4.4 en cas de compromission d'un compte admin à moindre privilège.

Recommandation : généraliser l'usage de `escapeHtml()` à tous les rendus `innerHTML` de l'admin.

SÉVÉRITÉ BASSE/MOYENNE**4.6 — Absence de headers de sécurité HTTP**

bootstrap/app.php · app/Http/Middleware/ · public/.htaccess

Aucune trace de middleware ou configuration ajoutant CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy ou Strict-Transport-Security. Le .htaccess public ne configure que le rewrite Laravel standard.

Recommandation : ajouter un middleware global (ou configuration serveur/proxy) définissant ces headers, notamment une CSP adaptée au contenu Blade + Vite.

SÉVÉRITÉ BASSE**4.7 — Confiance globale envers les proxys**

bootstrap/app.php, ligne 16

```
$middleware->trustProxies(at: '*');
```

Fait confiance à tous les proxys pour les headers X-Forwarded-* (IP, host, proto), ce qui affecte request()->ip() — stocké tel quel dans Message::ip_address via ContactController::store(). Un attaquant en amont d'un reverse proxy mal configuré pourrait usurper son IP journalisée. Compromis courant sur les plateformes PaaS, mais à restreindre idéalement aux IP du proxy connu.

SÉVÉRITÉ BASSE**4.8 — Route de test oubliée, vue manquante**

routes/web.php, lignes 154-157

```
Route::get('/test-api', function () {  
    return view('test_api');  
})->name('test.api');
```

La vue resources/views/test_api.blade.php n'existe pas dans le projet. Cette route publique est morte et provoquera une erreur 500 si elle est appelée. Avec APP_DEBUG=false (cas actuel) elle ne fuira pas de stack trace, mais reste un vestige de debug à supprimer.

POINTS POSITIFS 4.9 — Bonnes pratiques constatées

- Aucune injection SQL brute détectée : toutes les requêtes passent par Eloquent avec binding automatique.
- Aucun secret ni API key en dur trouvé dans le code applicatif (`resources/js` , `resources/views` , `app/`).
- Mass assignment maîtrisé : les contrôleurs construisent explicitement les tableaux validés avant `create()` / `update()` , pas d'exposition directe de `is_admin` trouvée.
- Verrouillage anti-bruteforce sur le login (`ThrottlesLogins`) et vérification email personnalisée avec `throttle`.
- Cookies de session `HttpOnly` par défaut ; `.env` réel correctement exclu du contrôle de version via `.gitignore` .
- Aucun endpoint d'upload de fichier trouvé — pas de surface d'attaque liée aux uploads non sécurisés à ce stade.

5. Performance

5.1 Absence de pagination sur plusieurs listings

`ServiceController::index()` et `ProductController::index()` renvoient l'intégralité des tables via `->get()` sans pagination (seul `ProjectController::index()` pagine, 9 par page). Côté admin, seul `Admin/Api/MessageController::index()` pagine (50/page) ; les autres endpoints Admin/Api (services, projets, partenaires) renvoient tout sans limite. Impact limité tant que le catalogue reste petit, mais absence de garde-fou en cas de croissance des données.

5.2 Absence d'index en base sur les colonnes de filtrage/tri fréquentes

Seules les colonnes `slug` ont un index (unique) dans les migrations. Aucun index sur `is_active` , `status` , `category` , `order` , `is_featured` , bien que ces colonnes soient systématiquement utilisées dans les scopes `active()` , `ordered()` , `featured()` , `byCategory()` .

5.3 Pas de couche de cache applicative

Aucun usage de `Cache::` détecté dans les contrôleurs : les pages publiques interrogent la base à chaque requête sans mise en cache, alors que le contenu d'un site vitrine change rarement. `CACHE_DRIVER=file` en `.env` — acceptable à faible trafic, non scalable au-delà.

5.4 Serveur de développement utilisé en production

```
nixpacks.toml : php artisan serve --host=0.0.0.0 --port=${PORT:-8000}
```

`php artisan serve` est le serveur de développement intégré de Laravel : mono-thread, sans gestion de process, performances limitées sous charge. Non recommandé pour un déploiement en production (préférer PHP-FPM + Nginx/Caddy, ou Octane).

5.5 Build front-end

Configuration Vite/Laravel par défaut (minification et code-splitting assurés en mode `build`). L'application étant majoritairement du Blade avec un point d'entrée JS unique, le besoin de code-splitting avancé reste faible. Pas d'optimisation d'images (srcset/lazy-loading systématique) constatée dans les vues.

6. Qualité, tests et manquements

6.1 Tests — suite non fonctionnelle

`tests/Feature/ExampleTest.php` et `tests/Unit/ExampleTest.php` sont du code de démonstration non adapté à l'application réelle :

- `test_api_health_endpoint()` appelle `/api/health`, route inexistante (seul `/up` existe par défaut) → test cassé.
- `test_about_page_loads_successfully()` appelle `/about` alors que la route réelle est `/a-propos` → test cassé.
- `test_contact_form_submission()` attend un message de session en anglais, alors que `ContactController::store()` renvoie un message en français → assertion incorrecte.

Aucun test ne couvre les contrôleurs Admin/Api, la validation, l'autorisation (AdminMiddleware) ni les modèles : couverture réelle proche de zéro.

6.2 Linting / Formatting

- `prettier.config.js` présent, script `format / format:check` opérationnel.
- `npm run lint` **référence ESLint avec plugins Vue/TypeScript installés, mais aucun fichier de configuration ESLint (`.eslintrc*` / `eslint.config.js`) n'existe → le script échoue immédiatement.**
- `laravel/pint` installé côté PHP mais non intégré à un script composer (pas de `composer lint`).
- `.editorconfig` présent — bon point pour la cohérence de style basique.

6.3 Documentation

`README.md` présent (base). Aucun `CLAUDE.md` ni documentation d'architecture, des endpoints API admin, ou des rôles utilisateurs. Quelques commentaires en français dans le code expliquant certains choix (proxy, reverse-proxy Dokploy/Traefik), mais rien de structuré.

6.4 Gestion des erreurs et logging

`withExceptions()` dans `bootstrap/app.php` est vide (aucune personnalisation du rendu d'erreurs ni du filtrage des données sensibles dans les logs). `LOG_LEVEL` non défini explicitement → niveau `debug` par défaut, risque de fuite d'informations sensibles dans les logs en production si non changé. Aucun service de monitoring d'erreurs externe (Sentry, Bugsnag, Flare) configuré.

6.5 CI/CD — absent

Aucun dossier `.github/workflows`, aucun pipeline détecté. Le script `npm run ci ( lint && test && type-check && audit )` existe mais n'est déclenché par aucune automatisation, et échouera de toute façon tant que la configuration ESLint manque (§6.2).

6.6 Fichiers morts et dette technique

- `app/Models/ServiceResource.php` , `StoreServiceRequest.php` , `UpdateServiceRequest.php` : fichiers PHP vides (0 octet) dans le namespace `App\Models` , jamais implémentés.
- Dépendances Vue/TypeScript/Vitest installées mais quasiment inutilisées (aucun composant `.vue` trouvé dans `resources/js`) — poids mort dans le bundle de dev.

7. Tableau de synthèse des priorités

Priorité	Constat	Fichier(s)
HAUTE	CSRF désactivé sur toute l'API admin	bootstrap/app.php:18
HAUTE	APP_KEY vide en .env réel (chiffrement session cassé)	.env:10
MOYENNE	Fallback admin par email en dur (porte dérobée potentielle)	AdminMiddleware.php:27, LoginController.php:17
MOYENNE	Stored XSS possible via contenu non sanitisé	service/project/product-detail.blade.php
MOYENNE	Aucun header de sécurité HTTP (CSP, HSTS, X-Frame-Options)	global
MOYENNE	Serveur de développement (<code>php artisan serve</code>) utilisé en production	nixpacks.toml
BASSE/MOY.	innerHTML non échappé dans plusieurs vues admin	admin/products.blade.php, admin/projects.blade.php, admin/services.blade.php
BASSE	Suite de tests cassée/non représentative	tests/Feature/ExampleTest.php
BASSE	Configuration ESLint manquante malgré script <code>lint</code>	racine projet
BASSE	Pas de pipeline CI/CD	—
BASSE	Pas de pagination/index DB sur plusieurs listings	ServiceController.php, ProductController.php, migrations
BASSE	Route <code>/test-api</code> morte (vue manquante)	routes/web.php:154
BASSE	Fichiers modèles vides (dette technique)	app/Models/ServiceResource.php, etc.

8. Plan de remédiation recommandé

Court terme (avant mise en production)

- Retirer `api/admin/*` de l'exception CSRF dans `bootstrap/app.php`, ou migrer l'admin vers Sanctum SPA.
- Générer un `APP_KEY` valide par environnement (`php artisan key:generate`) et vérifier tous les secrets requis avant déploiement.
- Supprimer le fallback d'email admin en dur dans `AdminMiddleware` et `LoginController`.
- Remplacer `php artisan serve` par un stack de production (PHP-FPM + Nginx/Caddy, ou Laravel Octane).
- Supprimer la route morte `/test-api`.

Moyen terme

- Sanitiser le HTML riche affiché via `{!! !!}` (HTMLPurifier ou équivalent) et généraliser `escapeHtml()` côté admin.
- Ajouter un middleware de headers de sécurité HTTP (CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy).
- Restreindre `trustProxies` aux IP du proxy connu plutôt que `'*'`.
- Ajouter des index DB sur les colonnes `is_active`, `status`, `category`, `order` et paginer systématiquement les listings publics et admin.
- Mettre en place une couche de cache pour les pages/vitrines à faible fréquence de changement.

Continu / outillage

- Réécrire la suite de tests pour couvrir réellement les routes, l'autorisation admin et la validation.
- Ajouter une configuration ESLint fonctionnelle ou retirer l'outillage Vue/TS/Vitest inutilisé pour réduire la dette.
- Mettre en place un pipeline CI/CD (GitHub Actions ou équivalent) exécutant lint, tests, `composer audit` et `npm audit` à chaque changement.
- Définir explicitement `LOG_LEVEL` en production et brancher un service de monitoring d'erreurs.